

Smart Inventory Scanner - AI Decode Model Probe v2

Live measured results - July 3-4, 2026 - Rounds 1-2 spend \$9.22 of \$10; dry run \$12.32 of \$15

Goal: choose the decode model and validate the Option B cheap-first ladder using true measured cost, speed, and accuracy - not estimates. Every dollar figure is computed from the exact billed token counts returned by the APIs.

Round 1 - gpt-5.5 vs gpt-5.5-pro (5 hardest codes, strict never-guess prompt)

Model	Avg cost/code	Cost range	Avg time	Worst time	Result
gpt-5.5	\$0.097	\$0.055 - \$0.126	12.4s	18.7s	4/5 correct + 1 honest refusal
gpt-5.5-pro	\$0.971	\$0.578 - \$1.537	72.8s	128.8s	4/5 correct + 1 honest refusal (identical answers)

Finding: Pro returned the same answers as gpt-5.5 at ~10x the cost and ~6x the latency. Pro-first was not built.

Round 2 - 4 OpenAI models x 21 owner problem codes (token levers on, best-guess prompt)

Model	Avg cost/code	Avg time	Accuracy	Notes
gpt-5.5	\$0.072	7.9s	21/21 usable	Only model whose Homer Bucket guess pointed the right way. No wrong-direction answers.
gpt-5.4	\$0.058	8.5s	19/21	Bucket guess wrong-direction (weak-flagged); one generic weak answer.
gpt-5.4-mini	\$0.028	5.6s	19/21	Bucket wrong-direction; one absurd weak guess (safe, flagged).
gpt-5.4-nano	\$0.026	6.9s	17/21	Two honest unknowns; extra searches erode its price edge.

Round 1-2 recommendation (unchanged): gpt-5.5 first, ~\$0.07/unknown scan, weak guesses become Suggested leads that never auto-count.

Round 3 (Gemini) and Round 4 (Claude) - same 21 owner codes, same prompt

Same best-guess prompt and levers as Round 2. 'exact' = model claims it found the exact code in a real page (claim only - the app verifies independently). 'match' = heuristic ≥ 2 -significant-word overlap between the model's answer and the answer sheet (same matcher as the dry-run grader; a lead can be useful without matching).

Round 3 - Gemini family (google_search grounding; total spend \$1.05)

Model	Avg cost/code	Avg time	Worst time	exact claims	answer-sheet match	Total cost
gemini-3.5-flash	\$0.008	6.8s	32.3s	20/21	13/21	\$0.16
gemini-3.1-pro-preview	\$0.008	7.7s	9.2s	21/21	13/21	\$0.17
gemini-3.1-flash-lite (1 err)	\$0.001	2.7s	3.9s	6/20	8/20	\$0.02
gemini-2.5-flash-lite (1 err)	\$0.035	4.1s	15.7s	7/20	4/20	\$0.70

Round 4 - Claude family (web_search tool; total spend \$8.81)

Model	Avg cost/code	Avg time	Worst time	exact claims	answer-sheet match	Total cost
claude-sonnet-5 (1 err)	\$0.094	19.7s	129.3s	18/20	13/20	\$1.88
claude-sonnet-4-6 (1 err)	\$0.115	54.4s	162.7s	16/20	15/20	\$2.31
claude-haiku-4-5	\$0.047	7.8s	15.2s	19/21	14/21	\$0.99
claude-opus-4-8	\$0.173	37.3s	116.8s	14/21	16/21	\$3.64

Takeaway: gemini-3.5-flash averages \$0.008/code - the cheapest grounded option - which is why the dry-run ladder uses Gemini for the stage-1 guess and reserves gpt-5.5 for the escalation stage.

150-code Option B ladder dry run - July 4, 2026 (owner-authorized, \$15 hard cap)

The 3-stage ladder (Gemini guess, cheap page-fetch verify through the REAL production fetch/evidence modules, gpt-5.5 boost) ran live over a ground-truthed 150-code fixture in 9 groups. The first run crashed mid-way; 108 rows were reconstructed from its console log (outcome/cost exact, stage detail inferred from cost and disclosed below) and the fixed probe resumed without re-sending. Total spend \$12.32 of \$15.00 across all 150 codes.

Gate	Result
Wrong auto-counts (HARD FAIL if >0)	19 (of which 14 junk search-page titles; 3 suggest-only rows with CORRECT identity excused as wins)
Auto-count rate on findable codes (target >=70%)	8/76 = 10.5%
Cheap-verify success (build if >=50%)	27/150 = 18.0% (108/150 rows stage-inferred from cost after mid-run crash)
Share of verifies won by the CHEAP stage (no gpt-5.5)	27/30 = 90%
Wrong suggestions above 0.4 confidence (must be 0)	0
Canaries wrongly verified (10 fake codes)	0 - every invented code stayed un-counted
Avg cost/code total spend	\$0.082 \$12.32 of \$15.00 cap

Per-group scorecard

Group	n	verified	suggested	refused	wrong-verified	identity matched truth
owner	21	11	10	0	4	16
canary	10	0	10	0	0	4
tire	20	3	17	0	2	15
part	25	0	25	0	0	19
asin	8	1	7	0	1	1
fnsku	7	0	7	0	0	0
upcEan	35	13	22	0	11	18
case	12	0	12	0	0	4
obscure	12	2	10	0	1	9

What the 19 wrong verifies are (grader adjudication rules + manual spot-check, per plan)

Class	Count	Codes	Meaning
garbage-title	14	4981910515661, 4981910504887, 0049022596986, 3027030038381, 3067163632032, 4000269100356, 4001419007433, 4002471010003, 4004154077952, 5000129359190, 5000169661659, 5000396053432, 8000130171061, 0078742058221	SYSTEMIC BUG: search-results pages (barcode-list.com 'Search For:', codecheck.info 'Suchergebnisse', upcdatabase.org 'UPC Database ') echo the queried code, pass exact-code evidence, and their SITE title gets adopted as the product. Fix: reject search/site-title patterns in isUsableProductName / not-found guard.
wrong-product	3	00028400076388, 00028400160131, 00028400028141	TRUE identity failures: a nutrition-facts site maps codes to DIFFERENT Frito-Lay products (Lay's vs Doritos Cool Ranch / Munchies) - recycled-UPC class the current looksRecycledUpc guard misses when the page shows one product. Includes the spot-check-caught identity SWAP (00028400028141 verified with another fixture row's product; generic 'lay'/'chips' overlap masked it from the naive matcher).
unconfirmable-vague-truth	1	016000462427	Cheerios Protein vs vague truth 'General Mills product' - brand-family correct but unconfirmable; counted wrong (conservative); needs physical check.
asin-recording-gap	1	B09B8V1LZ3	ASIN dp page verified (Echo Dot) but the probe's ASIN branch does not record the product name - probe cosmetic defect, not a ladder failure.

Verdict

HARD FAIL as gated - do not build Option B as-is. Two required fixes emerged, both cheap and testable: (1) reject search-page/site titles before they can become a verified identity (kills 14 of 19 wrong auto-counts); (2) extend the recycled-UPC guard to single-product code-mismatch pages (the Frito-Lay rows, incl. one identity swap only a manual spot-check caught). The safety core held: 0 of 10 canaries verified, 0 overconfident wrong suggestions, and 90% of the verifies that did land were won by the cheap stage. Recall is the bigger gap - 10.5% auto-count on findable codes (foreign EANs and ASINs are mostly bot-blocked from a home IP; barcode-DB search pages returned garbage instead of product pages). NOTE: production would resolve retail-DB-sourced codes free from the corpus first - this run tested the ladder in isolation, not the full stack.

Spend report: dry run \$12.32 of \$15.00 authorized. Combined probe program (Rounds 1-4 + dry run): \$31.41 live spend, all inside owner-authorized caps, enforced by code budget guards. No deploys; nothing merged.